

Day 10 — Final Assessment & Capstone

Your simple, friendly guide for the last day

COMING FROM	KnockoutJS / .NET (C#) → moving to React + TypeScript
TODAY'S GOAL	Prove you can do it on your own: plan the architecture, build the Employee Management Portal end-to-end, debug it calmly, review your own code, and refactor it clean.

How to use this: Read **Section 1 (Learning Plan)** for each idea with a small example. Then read **Section 2 (Detailed Notes)** slowly, top to bottom — one read should make it click. Keep **Section 3 (Common Mistakes)** handy while you practise, and copy **Section 4 (Concept Notes)** by pen for recall & interviews.

1 Learning Plan

Welcome to the **last day**. Today is not a scary exam — it is your victory lap. You take everything from Days 1-9 and put it together into one real app: the **Employee Management Portal**. Think of it like the end of your onboarding sprint at a new job: you are not learning something brand-new, you are *proving to yourself* that you can build production React + TypeScript on your own. Skill check, not a fear test.

The day has five parts. Each one is a skill an interviewer or a senior dev would look for. We go through all five, then fold in some extra topics at the end.

Part A — The Five Assessment Areas

1. Architecture — Plan Before You Code

Before you type a single line, sketch the app on paper. Two questions answer 80% of React design: **"What components do I need?"** and **"Where does each piece of state live?"** In React, data flows **down** through props, and things that happen (a click, a typed letter) flow **up** through callback functions. ("State" = data that can change over time and makes the screen re-draw. "Props" = read-only values a parent passes to a child, like arguments to a function.)

A tiny plan for our portal looks like this:

```
<App>                                (routing lives here)
|
|— <EmployeeListPage>                 route: /employees
|   |— SearchBar                     local state: "query" text
|   |— EmployeeTable                 gets employees + loading/error as props
|   |— Pagination                     local state: "page" number
|
|— <EmployeeDetailPage>               route: /employees/:id
|   |— reads :id from URL, fetches one employee
|
|— <EmployeeFormPage>                 routes: /employees/new and /employees/:id/edit
    |— react-hook-form + zod (the form owns its own state)
```

Where does state live? – one rule each:

- local useState → only ONE component uses it (SearchBar's query text)
- lifted state → two SIBLINGS need it → move it up to their shared parent
- Context → MANY components deep down need it (current user, theme)
- useReducer → ONE piece of state with many related fields / action types

Data flow: props go **down** (parent hands data to child); events go **up** (child calls a function the parent gave it). That is the whole game.

- **.NET / KnockoutJS contrast:** This is exactly like designing your layers and classes before coding — which class owns which data, what the DTOs look like, and who calls whom. Deciding "local vs Context" is like deciding "private field vs shared service."
 - **You'll be able to...** draw a component tree and confidently say where every piece of state belongs *before* writing code.
-

2. Coding the Capstone — The Employee Management Portal

Now you build it end-to-end, on your own. The portal has: a **list** with **search + pagination**, an **add/edit form** with validation (react-hook-form + zod), an **employee detail page** with routing, a **typed API/service layer** (axios in one place), and proper **loading / error / empty** states so the screen never looks broken. ("Service layer" = one file that owns all the HTTP calls, so your components never touch axios directly — like a C# repository or `HttpClient` wrapper.)

First, the typed service — all HTTP lives here:

```
// services/employeeService.ts
import axios from "axios";

export interface Employee {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
}

interface EmployeeListResponse {
  users: Employee[];
  total: number;
}

const api = axios.create({ baseURL: "https://dummyjson.com" });

export async function getEmployees(
  search: string,
  page: number,
): Promise<EmployeeListResponse> {
  const limit = 10;
  const skip = (page - 1) * limit;
  const url = search ? "/users/search" : "/users";
  const res = await api.get<EmployeeListResponse>(url, {
    params: { q: search, limit, skip },
  });
  return res.data;
}
```

Then a **custom data hook** that calls the service and hands your component clean, typed state (data + loading + error). Your component just reads these four values:

```

// hooks/useEmployees.ts
import { useEffect, useState } from "react";
import { getEmployees, type Employee } from "../services/employeeService";

export function useEmployees(search: string, page: number) {
  const [data, setData] = useState<Employee[]>([]);
  const [total, setTotal] = useState(0);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let active = true; // ignore the result if inputs change first

    setLoading(true);
    setError(null);

    getEmployees(search, page)
      .then((res) => {
        if (!active) return;
        setData(res.users);
        setTotal(res.total);
      })
      .catch(() => {
        if (active) setError("Could not load employees.");
      })
      .finally(() => {
        if (active) setLoading(false);
      });

    return () => {
      active = false; // cleanup runs before the next fetch
    };
  }, [search, page]);

  return { data, total, loading, error };
}

```

In the component you then handle all three states plainly: if `loading` show a spinner, if `error` show a retry message, if `data.length === 0` show "No employees found", otherwise show the table. The add/edit form uses zod for the rules and react-hook-form for the wiring (same pattern you learned on Day 6).

- **.NET / KnockoutJS contrast:** The service layer is your `HttpClient` /repository; the custom hook is a small view-model that exposes bound data. In Knockout you wrote `ko.observable` and manual subscriptions — here `useState` + `useEffect` do that binding for you.
 - **You'll be able to...** build a full, typed CRUD screen with search, paging, routing, and honest loading/error/empty states — by yourself.
-

3. Debugging — The Calm Method

When something breaks, do not panic or randomly change lines. Follow five calm steps: **reproduce** (make the bug happen on purpose) → **isolate** (find the smallest place it happens) → **inspect** (open React DevTools to see a component's props/state/hooks, and the browser **Network** tab for failed calls + the **Console** tab for errors) → **fix** (change one thing) → **verify** (confirm it is gone and nothing else broke). ("React DevTools" = a browser add-on that lets you click any component and watch its live props and state.)

The classic React bugs and their quick fixes:

- **"Each child in a list should have a unique key" warning** → add a stable `key={item.id}` to the element you `.map()` over. Never use the array index if the list can reorder.
 - **Infinite re-render / screen freezes** → you called `setState` during render, or a `useEffect` has bad dependencies (or none). Only set state inside events or effects, and list the real dependencies.
 - **Stale data / effect not re-running** → a missing `useEffect` dependency (a "stale closure" — the effect captured an old value). Add the value to the dependency array.
 - **State change ignored** → you mutated state directly (`arr.push(x)` or `obj.name = " ... "`). Always make a **new** object/array: `setItems([ ... items, x])` .
 - **.NET / KnockoutJS contrast:** React DevTools is your Visual Studio debugger + Watch window — you inspect live values instead of guessing. "Reproduce → isolate → fix → verify" is the same discipline you already use with breakpoints.
 - **You'll be able to...** calmly find and fix a React bug using DevTools and the Network/Console tabs, instead of trial-and-error.
-

4. Review — Self-Review (Before You "Submit")

Great engineers read their own code before asking anyone else to. Run this checklist on your capstone like you would review your own Pull Request before requesting a reviewer:

Self-review checklist

- [] Types: real interfaces, NO `any` left anywhere
- [] Every fetch screen has loading + error + empty states
- [] No leftover console.log, no secrets/keys in the code
- [] Names are clear (getEmployees, not doStuff / temp / data2)
- [] Components are small and do ONE thing
- [] Every .map() list has a stable key
- [] Accessibility basics: <label> on inputs, alt on images, button is a <button>
- [] Edge cases handled: empty list, one item, very long name, failed request
- [] State is updated immutably (new object/array, never mutate)
- [] No unused imports / dead code

- **.NET / KnockoutJS contrast:** This is reviewing your own PR before requesting review — the same self-check pass you would run in a code review, just done by you first.
- **You'll be able to...** catch your own mistakes with a checklist, so your code is ready before anyone else sees it.

5. Refactoring — Small, Safe Steps

Refactoring means **improving the code without changing what it does**. The trick: tiny steps, and verify after each one. Common moves are: **extract a component**, **extract a custom hook**, **remove duplication (DRY)**, and **rename for clarity**. Here filtering logic that was buried inside a component gets pulled into a reusable hook — behavior is identical, but now it is testable and reusable:

```
// BEFORE – the filtering logic sits inline inside the component
function EmployeeList({
  employees,
  query,
}: {
  employees: Employee[];
  query: string;
}) {
  const visible = employees.filter(
    (e) =>
      e.firstName.toLowerCase().includes(query.toLowerCase()) ||
      e.lastName.toLowerCase().includes(query.toLowerCase()),
  );
  return (
    <ul>
      {visible.map((e) => (
        <li key={e.id}>{e.firstName}</li>
      ))}
    </ul>
  );
}
```



```
// AFTER – same behavior, logic pulled into a reusable, testable hook
function useEmployeeSearch(employees: Employee[], query: string) {
  const q = query.toLowerCase();
  return employees.filter(
    (e) =>
      e.firstName.toLowerCase().includes(q) ||
      e.lastName.toLowerCase().includes(q),
  );
}

function EmployeeList({
  employees,
  query,
}: {
  employees: Employee[];
  query: string;
}) {
  const visible = useEmployeeSearch(employees, query);
  return (
    <ul>
      {visible.map((e) => (
        <li key={e.id}>{e.firstName}</li>
      ))}
    </ul>
  );
}
```

- **.NET / KnockoutJS contrast:** This is ReSharper / Roslyn "Extract Method" and "Rename" — safe, mechanical improvements that keep behavior identical. Do one, re-run, confirm, repeat.
- **You'll be able to...** clean up messy code in small safe steps, checking after each step that nothing broke.

Part B — Extra Topics

Self-Assessment Checklist (Days 1-9)

Tick each one honestly. Every "yes" is a skill you now own:

- Day 1 – Can you use ES6+ (arrow fns, destructuring, spread, map/filter, async/await)?
- Day 2 – Can you type variables, props, and API shapes with interfaces (no `any`)?
- Day 3 – Can you build a component with JSX and pass typed props?
- Day 4 – Can you add and update local state with useState (immutably)?
- Day 5 – Can you fetch data with useEffect + axios and show loading/error?
- Day 6 – Can you add routes (react-router-dom) and a validated form (react-hook-form + zod)?
- Day 7 – Can you build a reusable component and a custom hook?
- Day 8 – Can you use Context, useReducer, memo/useMemo/useCallback, lazy + error boundaries?
- Day 9 – Can you use Git/PRs, a feature-folder layout, and a typed service layer?

How to Demo Your Work in 2 Minutes

Like a stand-up or an interview walk-through — keep it to four beats:

1. **What it does** — "It is an Employee portal: list, search, paging, add/edit with validation, and a detail page."
2. **How it's structured** — "Feature folders; components read data from a `useEmployees` hook; all HTTP lives in one typed service."
3. **One thing you're proud of** — e.g. "Clean loading/error/empty states, no `any` anywhere."
4. **One thing you'd improve** — e.g. "I'd add caching and a few tests." (Naming a next step shows maturity, not weakness.)

Performance Quick Wins (use only when needed)

Reach for these **only after** you notice a real slowdown — never by default:

- `React.memo` — skip re-rendering a child when its props did not change.
- `useMemo` — cache an expensive calculation between renders.
- `useCallback` — keep a function's identity stable so a memo'd child does not re-render.
- `lazy` + `Suspense` — load a heavy page/route only when the user opens it.

A Gentle Testing Mindset

You are not writing tests today, but think like a tester. For every screen ask: *what if the list is empty? what if the API fails? what if the name is very long? what if I click Save twice?* Handling those cases in your head (and in your code) is 90% of the value of tests — you are just doing it with your eyes for now.

.NET Contrast — Quick Cheat Sheet

Today's activity	Feels like in .NET land
The assessment itself	A technical interview / a work sprint

Today's activity	Feels like in .NET land
Architecture planning	Designing your layers and class structure
Debugging with React DevTools	The Visual Studio debugger + Watch window
Refactoring	ReSharper / Roslyn refactors
Self code-review	Reviewing your own PR before requesting review

That's the plan for Day 10. Plan it, build it, debug it calmly, review it yourself, then polish with small refactors. Do that and you have proven — to the only person who matters, you — that you can ship production React + TypeScript. Congratulations on reaching the final day.

2

Detailed Notes — Read Once, Understand Everything

The big idea first

Day 10 is the day the whole plan clicks into place. You are **not** learning a new library or a new API today. Instead, you are going to *prove* — to yourself — that you can take everything from Days 1–9 and use it to **plan, build, debug, review, and polish a real app on your own.**

The app is the **Employee Management Portal**. It pulls in a skill from almost every day: modern JS and TypeScript types (Days 1–2), components and props (Day 3), state with `useState` (Day 4), data fetching with a custom hook (Day 5), routing and validated forms (Day 6), reusable components and custom hooks (Day 7), Context / `useReducer` where needed (Day 8), and the production folder + service-layer patterns (Day 9).

Think of it like your first solo work sprint back in .NET. Nobody hands you a new framework — they hand you a ticket and say "build the feature end-to-end, then get it review-ready." The five areas below (Architecture, Coding, Debugging, Review, Refactoring) are exactly the five things a working React + TypeScript developer does every single day.

1. Architecture first — plan before you type

The biggest beginner mistake is to open a file and start typing components. Don't. Do what you already do in C#: **design the shape first**. Before writing a class, you decide your layers, your models, and who talks to who. Same here — you sketch the pages and components, then decide where each piece of state lives.

Step 1 — sketch the component tree (just on paper or in a comment). Boxes = components. Arrows in your head = data flowing.

```
<App>
├─ <AuthProvider>           (Context: who is logged in – app-wide)
│   └─ <Routes>
│       ├─ /employees       <EmployeeListPage>
│       │   ├─ <SearchBar>   (local state: the search text)
│       │   ├─ <EmployeeTable> (gets `employees` via props)
│       │   │   └─ <EmployeeRow> (props DOWN; onSelect event UP)
│       │   └─ <Pagination>  (props: page, total; event: onPageChange)
│       └─ /employees/:id    <EmployeeDetailPage>
│           └─ /employees/new <EmployeeFormPage> (react-hook-form + zod)
```

Step 2 — decide where each piece of state lives. This is the single most important architecture decision, and there's a simple rule for each choice:

- **Local `useState`** — only one component cares about it. A form input's text, an "is this dropdown open?" toggle. *Like a `private` field inside one class.*
- **Lifted state** — two sibling components need the *same* value, so you move it up to their nearest shared parent, then pass it down. *Like moving a field up to a parent class that both children read from.*
- **Context** — truly app-wide things: the logged-in user, the theme, the current language. *Like a singleton or a service you inject with DI so any layer can reach it — without threading it through every constructor.*
- **`useReducer`** — one piece of state that has **many related actions** (a multi-field form, a wizard, a shopping cart: add / remove / clear / update). *Like a small state machine or a command handler*
- **`switch`** — *all the "how state changes" logic lives in one predictable place.*

Step 3 — remember the golden rule of data flow: props go DOWN, events come UP. A parent hands data to a child through props. The child never reaches up to change the parent — instead the parent hands it a **callback** (a function prop), and the child *calls* it to say "hey, something happened." This is the same idea as passing a delegate/ `Action<T>` into a method so the caller decides what happens.

```
type Employee = { id: number; name: string; department: string };

// Parent OWNS the state. The child just receives data + a callback.
// Data flows down via `employee`; the event flows up via `onSelect`.
function EmployeeRow({ employee, onSelect }: {
  employee: Employee;
  onSelect: (id: number) => void;
}) {
  return (
    <button onClick={() => onSelect(employee.id)}>
      {employee.name} - {employee.department}
    </button>
  );
}
```

Step 4 — lay out the folders by feature (this is the Day 9 pattern). Everything about "employees" lives together, so a new teammate can find it instantly:

```

src/
├─ features/
│  └─ employees/
│     ├─ api/employeeService.ts    (typed axios calls – the "layer" that talks to the
server)
│     ├─ hooks/useEmployees.ts    (data + loading + error)
│     ├─ components/EmployeeTable.tsx
│     ├─ components/EmployeeRow.tsx
│     ├─ pages/EmployeeListPage.tsx
│     ├─ pages/EmployeeDetailPage.tsx
│     ├─ pages/EmployeeFormPage.tsx
│     └─ types.ts                (the shared Employee type)
├─ shared/                      (reusable Button, Input, useDebounce...)
└─ App.tsx                      (routes + providers)

```

Ten minutes of planning here saves you an hour of untangling later — exactly like sketching your class diagram before writing C#.

2. Build the capstone step by step

Now you build — but **one slice at a time, bottom to top, checking each slice works before moving on**. This is the same instinct as building a feature vertically in .NET: repository → service → controller → view, testing each layer as you go. Never try to build all five pages at once.

Here's the order and why:

1. **The typed API/service layer first** (`employeeService.ts`). One small file of typed `axios` functions. This is your "data access layer." Get it returning real data before you build any UI.
2. **The custom hook** (`useEmployees`). It wraps the service call and hands your components three things every screen needs: **the data, a loading flag, and an error message**. This keeps fetching logic out of your components.
3. **The list page**. Search box, pagination, and — importantly — the three UI states: **loading, error, and empty** (no results), plus the happy path where you show the list.
4. **The add/edit form**. `react-hook-form` for the inputs + `zod` for validation (the rules live in one schema, and TypeScript infers the form's type from it).
5. **The detail route**. `/employees/:id` reads the id from the URL and shows one employee.

Here is the representative piece — the `useEmployees` hook. Read it slowly; it uses `useState`, `useEffect`, the typed service call, cleanup, and the loading/error pattern all at once. This one file is basically Days 4, 5, and 9 combined:

```

import { useEffect, useState } from "react";
import axios from "axios";

export type Employee = {
  id: number;
  firstName: string;
  lastName: string;
  department: string;
};

type EmployeesResponse = { users: Employee[]; total: number };

type UseEmployeesResult = {
  employees: Employee[];
  total: number;
  loading: boolean;
  error: string | null;
};

// One hook = data + loading + error, ready for any component to use.
export function useEmployees(search: string, page: number): UseEmployeesResult {
  const [employees, setEmployees] = useState<Employee[]>([]);
  const [total, setTotal] = useState(0);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    const controller = new AbortController(); // lets us cancel if inputs change fast
    setLoading(true);
    setError(null);

    axios
      .get<EmployeesResponse>("https://dummyjson.com/users/search", {
        params: { q: search, limit: 10, skip: page * 10 },
        signal: controller.signal,
      })
      .then((res) => {
        setEmployees(res.data.users);
        setTotal(res.data.total);
      })
      .catch((err) => {

```

```

    if (!axios.isCancel(err)) setError("Could not load employees.");
  })
  .finally(() => setLoading(false));

  return () => controller.abort(); // cleanup: cancel the old request
}, [search, page]); // re-fetch whenever search text or page changes

return { employees, total, loading, error };
}

```

Now any page can just say `const { employees, loading, error } = useEmployees(search, page);` and render the right state. The component stays about **showing** things; the hook handles **getting** things. Clean separation — just like keeping your service logic out of your controller.

3. Debugging calmly

Something will break. That is normal — it breaks for senior devs too. The difference is they don't panic or guess randomly; they follow a calm loop. Use these **5 steps, in order**:

1. Reproduce — make the bug happen on purpose, every time. Write down the exact steps.
2. Isolate — narrow it down. Which component? Which line? Comment things out.
3. Inspect — LOOK, don't guess. Use DevTools + Network + Console (below).
4. Fix — the smallest change that fixes the real cause, not a band-aid.
5. Verify — redo step 1's steps. Bug gone? Did anything else break?

Your tools (you already know their .NET cousins):

- **React DevTools** (browser extension) = your **Visual Studio debugger + Watch window**. Click any component and you see its live **props, state, and hooks** — the actual values right now. This alone solves most "why is it showing the wrong thing?" bugs.
- **Network tab** = see the real HTTP request and response — the URL, the params, the status code (200? 404? 500?), and the JSON that came back. *Like Fiddler*. Great for "is it my UI or is the server sending junk?"
- **Console tab** = red error messages, yellow warnings, and anything you `console.log`. Read the first red error fully — it usually names the file and line.

Here are the **four classic React bugs** you'll almost certainly meet, each with the fix:

Bug A — missing `useEffect` dependency (a "stale" value). *Stale = old, out of date.*


```
// BEFORE: effect runs once; `search` is frozen at its first value forever
useEffect(() => {
  loadEmployees(search);
}, []); // missing `search` in the deps array

// AFTER: it re-runs whenever `search` changes, so it always uses the latest value
useEffect(() => {
  loadEmployees(search);
}, [search]);
```

Bug B — "Each child in a list should have a unique key." React needs a stable id per row to update lists efficiently.

```
// BEFORE: no key → console warning + subtle update bugs
{employees.map((e) => <EmployeeRow employee={e} onSelect={onSelect} />)}

// AFTER: use a STABLE id (not the array index, which shifts when the list changes)
{employees.map((e) => (
  <EmployeeRow key={e.id} employee={e} onSelect={onSelect} />
))}
```

Bug C — infinite re-render. Usually caused by an object/array being **recreated every render** and then used as an effect dependency. React sees a "new" object each time and re-runs forever.

```
// BEFORE: `filters` is a brand-new object on every render → effect fires endlessly
const filters = { department: "Sales" };
useEffect(() => {
  loadEmployees(filters);
}, [filters]);

// AFTER: depend on the primitive value (strings compare by value, not by reference)
const department = "Sales";
useEffect(() => {
  loadEmployees({ department });
}, [department]);
```

(The same rule bites you if you call `setState` directly in the render body — that triggers a render, which sets state, which renders... Only set state inside effects or event handlers.)

Bug D — mutating state directly instead of making a new copy. React decides whether to re-render by checking if the **reference** changed. If you edit the existing object, the reference is the same, so React may not update. *Immutable* = don't change the existing thing; make a fresh copy with the change.

```
// BEFORE: edits the object already in state, then passes the SAME array back
function promote(id: number) {
  const emp = employees.find((e) => e.id === id);
  if (emp) emp.department = "Management"; // mutating existing state
  setEmployees(employees);                // same reference → no re-render
}

// AFTER: build a NEW array with a NEW object for the one that changed
function promote(id: number) {
  setEmployees((prev) =>
    prev.map((e) => (e.id === id ? { ...e, department: "Management" } : e))
  );
}
```

4. Reviewing your own work

Before you'd ever open a PR for someone else to review, review it yourself first. Put on your "grumpy reviewer" hat and read your own code as if a stranger wrote it. This is exactly the habit of reading your own diff in the GitHub PR view before clicking "request review." Run down this checklist:

- [] Types: no `any`. Props, state, and API responses are all typed.
- [] States: loading, error, AND empty are all handled (not just the happy path).
- [] No leftover console.log. No hard-coded secrets / API keys / passwords.
- [] Names say what they mean (employeesLoading, not flag2 or data3).
- [] Components are small and do ONE job. A huge file? Split it.
- [] Every list has a stable key={id} (not the array index).
- [] Accessibility basics: every input has a <label>; clickable things are real <button>s.
- [] Edge cases: empty list, very long names, and a slow or failed network all look OK.

If any box fails, fix it now — it is far cheaper than after a teammate points it out.

5. Refactoring safely

Refactoring means **changing the *shape* of the code without changing what it *does***. The app behaves identically before and after — it's just cleaner. This is the same thing you did with ReSharper/Roslyn refactors in Visual Studio: "Extract Method," "Rename," "Inline." The rule that keeps you safe: **tiny steps, and re-check the app after each one**. Never do five refactors at once, or you won't know which one broke it.

Good small moves:

- **Extract a component** — a repeated block of JSX becomes one `<EmployeeRow />`.

- **Extract a custom hook** — copy-pasted fetch/loading/error logic becomes one `useEmployees` .
- **Rename** for clarity — `d` becomes `department` , `handleClick2` becomes `handleDelete` .
- **Delete dead code** — unused imports, commented-out blocks, an old variable nobody reads.

A tiny before/after — pulling duplicated data logic into the hook you already built:

```
// BEFORE: this page (and the detail page too) each copy-paste the same fetch block
function EmployeeListPage() {
  const [employees, setEmployees] = useState<Employee[]>([]);
  const [loading, setLoading] = useState(false);
  // ...the same useEffect + axios + setLoading logic, repeated in every page ...
  return <div>{/* render list */</div>;
}

// AFTER: one line. The data logic lives in ONE place and is reused everywhere.
function EmployeeListPage() {
  const { employees, loading, error } = useEmployees("", 0);
  if (loading) return <p>Loading...</p>;
  if (error) return <p>{error}</p>;
  if (employees.length === 0) return <p>No employees found.</p>;
  return <ul>{employees.map((e) => <li key={e.id}>{e.firstName}</li>)}</ul>;
}
```

Same behavior, less code, one source of truth. After each such step, click through the app once — if it still works, keep going; if not, undo that one step and try again.

In one line

If you can **plan** it, **build** it, **debug** it, **review** it, and **clean it up** — you can work as a React + TypeScript developer. That's the whole job, and today you did all five.

3 Common Mistakes to Avoid

These are the traps people fall into on a capstone, an assessment, or a technical interview. None of them are about "not knowing React" — they are about *how you work*. Read these once before you start, and again before you hit submit.

1. Coding before planning

❌ Open the editor and start typing components with no idea of the tree or where state lives. ✅ Spend 5 minutes sketching first: the component tree, where each piece of state lives, and how data flows.

In C# you don't start typing before you know your classes and layers — you picture the shape first. Same here. A tiny sketch on paper is enough:

```
App
└─ EmployeesPage      (owns: employees, search, page)  ← state lives here
    │   └─ SearchBar    (props: value; event: onChange)
    │   └─ EmployeeList (props: employees)
    │       └─ EmployeeRow (props: employee)
    └─ Pagination      (props: page, total; event: onPageChange)
```

Now you *know* the answer to "where does state live?" before you write a line. That one question is what most assessments are secretly testing.

2. Trying to build the whole app at once

❌ Wire up the list, the form, the detail page, and the API all in one big push — then nothing works and you can't tell why. ✅ Build one thin working slice at a time, in this order: **service** → **hook** → **list** → **form** → **detail**. Get each one working before the next.

Think of it like building your app one layer at a time and running it after each: data access first, then a hook that uses it, then the screen. If something breaks, you know it's the slice you just added — not a mystery buried in 500 lines.

Step 1	service	getEmployees() returns typed data	→ test in isolation
Step 2	hook	useEmployees() calls the service	→ returns data/loading/error
Step 3	list	render the rows	→ see real data on screen
Step 4	form	add / edit with validation	→ save works
Step 5	detail	route to one employee	→ deep-link works

3. Only handling the "happy path"

✗ You only render the data. No spinner, no error message, and a blank screen when the list is empty. ✓

Every screen that loads data handles **four** states: loading, error, empty, and data.

The empty state is the one everyone forgets. "No employees found" is a real thing a user will see (especially after a search filters everything out), so design for it.

```
if (isLoading) return <Spinner />;
if (error) return <ErrorBanner message={error} />;
if (employees.length === 0) return <EmptyState label="No employees found" />;
return <EmployeeList employees={employees} />;
```

4. Putting state in the wrong place

✗ Everything crammed into one giant component, OR reaching for Context to share a value only one component uses. ✓ Match the state to how far it needs to travel:

- Used by **one** component → `useState` right there.
- Shared by a **parent and its children** → lift it to the closest common parent, pass down as props.
- Truly **app-wide** (logged-in user, theme) → Context.
- **Complex updates** with many related fields → `useReducer`.

Context for a one-off value is like making a field `static` on a global class just so two methods can read it — it works, but now everything is coupled and hard to move. Keep state as *local* as you can.

5. Ignoring TypeScript / ESLint, or reaching for `any`

✗ Red squiggles everywhere, warnings ignored, and `any` sprinkled around "just to make it run." ✓
Treat a type error like a failing build — fix it, don't silence it. `any` switches off the very safety you came here for.

`any` is like casting everything to `object` in C# and hoping for the best: the compiler stops helping you, and a typo sails straight through to runtime.

```
// ✗ any throws away type safety - a typo like "employeez" is NOT caught
const res: any = await api.get("/employees");
res.data.employeez.forEach((e: any) => console.log(e.firstName));

// ✓ type the response - the compiler catches the typo for you
type EmployeesResponse = { employees: Employee[] };
const res = await api.get<EmployeesResponse>("/employees");
res.data.employees.forEach((e) => console.log(e.firstName));
```

6. Panicking while debugging

❌ Something breaks, so you change random lines and refresh over and over, hoping it fixes itself. ✅

Run the calm loop every time: **reproduce** → **isolate** → **inspect** → **fix** → **verify**.

- **Reproduce** — find the exact steps that trigger it.
- **Isolate** — narrow down *which* component or call is wrong.
- **Inspect** — open **React DevTools** (see each component's props, state, and hooks) and the browser **Network** + **Console** tabs. This is your Visual Studio debugger + watch window for React.
- **Fix** one thing.
- **Verify** it's actually gone.

Most "React is broken" moments are really "my API returned a 404" or "this prop is `undefined`" — and the Network/Console tabs tell you that in seconds.

7. The classic React bugs

These four come up again and again. Learn to spot them on sight.

a) Missing `useEffect` dependencies → stale data

❌ The effect never re-runs when the value it reads changes. ✅ List every value the effect uses in the dependency array (the ESLint hooks rule will tell you which).

```
// ❌ empty deps: fetches once, never re-fetches when employeeId changes
useEffect(() => {
  void loadEmployee(employeeId);
}, []); // eslint warns: missing 'employeeId'

// ✅ re-runs whenever employeeId changes
useEffect(() => {
  void loadEmployee(employeeId);
}, [employeeId]);
```

b) No `key` on list items

❌ `map` without a `key` → React warns and can render the wrong rows after a change. ✅ Give each item a stable, unique `key` — use the id, never the array index.

```
// ❌ <EmployeeRow employee={e} />                                     ← "Each child needs a key"
// ✅
{employees.map((e) => <EmployeeRow key={e.id} employee={e} />)}
```

c) Infinite re-render

- ❌ Calling a state setter directly during render → render triggers a set, which triggers a render, forever.
- ✅ Only set state inside an event handler or an effect.

```
// ❌ setCount runs on every render → infinite loop
// function Bad() {
//   const [count, setCount] = useState(0);
//   setCount(count + 1);
//   return <span>{count}</span>;
// }

// ✅ set state in response to an event
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount((c) => c + 1)}>{count}</button>;
}
```

d) Mutating state directly

- ❌ Changing the existing array/object in place → React sees the same reference and may not re-render.
- ✅ Always create a new value (spread/map/filter).

```
// ❌ mutates in place – same reference, unreliable update
// employees.push(newEmp);
// setEmployees(employees);

// ✅ new array – React sees the change and re-renders
setEmployees((prev) => [...prev, newEmp]);
```

Unlike C# where you happily `list.Add(x)`, React only re-renders when the *reference* changes. Treat state as read-only and replace it.

8. Not self-reviewing before "submitting"

- ❌ It runs, so you ship it — leftover `console.log`s, commented-out code, and a hardcoded API URL still inside.
- ✅ Do a quick self-review first, exactly like reading your own PR before asking a teammate. Run down a checklist:

- [] No `any`; response types defined
- [] Loading + error + empty states on every data screen
- [] Keys on every list
- [] No leftover console.log or commented-out code
- [] API URL in config/env – not hardcoded in a component
- [] Clear names; components small and focused
- [] Handles edge cases (empty search, failed save)

A hardcoded URL or a stray `console.log` is the code-review equivalent of leaving a `TODO` and a `Console.WriteLine` in a PR — small, but it's the first thing a reviewer spots.

9. Refactoring everything at once

✗ "Let me clean this all up," a hundred lines change, and now nothing works — and you can't tell which change broke it. ✓ Refactor in tiny steps, and re-check after **each** one. Extract one component. Run it. Extract one hook. Run it. Rename one thing. Run it.

Refactoring means improving the code *without changing what it does* (like a ReSharper extract-method). Small steps keep that promise; big-bang rewrites quietly break behavior and you won't notice until it's too late.

10. In an interview or demo: can't explain *why*

✗ "I did it this way because... that's how the tutorial did it." ✓ Be ready to explain your choices in one plain sentence each.

The assessor rarely cares that it works — they care that you *understood* it. Practice answering:

- "Why does this state live here?" → *Both the list and the search box need it, so I lifted it to their shared parent.*
- "Why a service layer?" → *So the API details live in one place and my components stay about the UI — easy to change or mock later.*
- "Why a custom hook?" → *To reuse the loading/error/data logic across screens without copy-pasting.*

If you can say *why*, you've proven the thing the whole capstone is checking for.

4 Concept Notes — Recall & Interview (copy by pen)

This is your recall + interview sheet for Day 10 — the final assessment. Plain-English definitions with C#/.NET comparisons, plus a compact revision of the whole 10-day plan. Copy it by pen — writing it out helps it stick.

Day 10 — Assessment Concepts

Software Architecture / Planning a React App

Definition:

- Deciding the app's shape BEFORE coding: the component tree, where state lives, how data flows, and the folder layout.
- C#/.NET: like designing your Controller → Service → Repository layers before writing any code.

✅ Advantages:

- Fewer painful rewrites later; every file has an obvious home.

❌ Disadvantages / watch-outs:

- Over-planning a tiny app wastes time — plan just enough, then build.

💡 Interview tip:

- "How do you structure a React app?" → feature folders + one-way data flow + a service layer for API calls.

📄 Example:

- `features/employees/{components, hooks, api, types}` — group files by feature, not by file type.

Where State Lives (local vs lifted vs Context vs useReducer)

Definition:

- Put each piece of state at the LOWEST place that still lets everyone who needs it read it.
- Local `useState` — only one component needs it (an input's value).
- Lifted state — move it up to the nearest shared parent when 2+ children need it.
- Context — app-wide values many components read (current user, theme); avoids "prop drilling" (passing props through many layers just to reach the bottom).
- useReducer — one screen with many related state changes (a form/wizard).
- C#/.NET: like choosing a local variable vs a field vs a shared singleton — smallest scope that works.

✅ Advantages:

- Local state is simplest and fastest; Context kills prop drilling.

❌ Disadvantages / watch-outs:

- Don't reach for Context too early — it can trigger extra re-renders; lift state first.

 Interview tip:

- "Where should state live?" → as local as possible; lift when shared; Context for truly global; useReducer for complex related updates.

 Example:

- `const [q, setQ] = useState('')` (local search) vs `<UserContext.Provider>` (global user).

Data Flow (props down / events up / one-way)

Definition:

- Data flows DOWN through props; changes flow UP through callback functions. One direction only.
- C#/.NET / Knockout: like a parent passing values down and the child raising an event the parent handles — one-way, not Knockout's two-way observable binding.

 Advantages:

- Easy to trace — one source of truth; you always know who owns the data.

 Disadvantages / watch-outs:

- More wiring (pass callbacks down); deep trees cause prop drilling → switch to Context.

 Interview tip:

- "Why one-way data flow?" → predictable and easy to debug; state changes happen in one known place, not scattered everywhere.

 Example:

- `<EmployeeRow emp={e} onEdit={handleEdit} />` — data down (`emp`), event up (`onEdit`).

Debugging Method + React DevTools

Definition:

- A calm, repeatable process: reproduce → isolate → inspect → fix → verify.
- Reproduce — find the exact clicks/data that trigger it.
- Isolate — narrow to one component or one function.
- Inspect — React DevTools (props, state, hooks) + browser Network tab (API calls) + Console (errors).
- Fix — smallest change that solves the root cause, not the symptom.
- Verify — redo the repro steps; confirm it's gone and nothing else broke.
- C#/.NET: React DevTools = Visual Studio debugger + watch window, but for components/props/state.

 Interview tip:

- "How do you debug a React bug?" → reproduce, isolate, inspect with DevTools/Network/Console, fix the root cause, verify.

 Example:

- Network tab shows `401` — the bug is auth/token, not your component render.

Common React Bugs

Definition:

- A handful of classic mistakes cause most React bugs.
- Stale closure / missing deps — a callback or effect uses OLD state because a value is missing from the `useEffect` deps array.
- Missing `key` — lists need a stable unique `key`; without it React mis-matches rows ("Each child in a list should have a unique key").
- Infinite re-render — setting state during render, or an effect whose deps change on every render.
- Mutating state — changing state in place instead of making a copy; React sees the same reference and won't re-render.

✗ Disadvantages / watch-outs:

- Don't use the array index as `key` if the list can reorder — use a stable id.

💡 Interview tip:

- "Why do lists need keys?" → keys let React match old and new items so it updates the right rows instead of re-rendering all of them.

📄 Example:

- `setItems([ ... items, newItem])` — immutable add (new array), never `items.push()`.

Immutable Updates

Definition:

- Never change state in place; always create a NEW object/array with the change.
- C#/.NET: like returning a new `record` with `with { }` instead of mutating fields.

✅ Advantages:

- React compares by reference to detect change — a new reference guarantees a reliable re-render.

📄 Example:

- `setUser({ ... user, name: 'Ana' })` — copy, then override one field.

Self Code-Review Checklist

Definition:

- Reviewing your OWN code before anyone else — reading your PR diff before hitting "request review".
- Types: no `any`; props and API responses are typed.
- States: loading, error, AND empty handled — not just the happy path.
- Clean: no leftover `console.log`, no secrets/keys in code.
- Clear names; components small and single-purpose.

- Lists have stable `key` s.
- Accessibility basics: labels on inputs, real `<button>` s.
- Edge cases: empty list, very long text, failed request.
- C#/.NET: same as self-reviewing a PR / running ReSharper inspections before submitting.

Example:

- `console.log` — search and remove all debug logs before you commit.

Refactoring (behavior-preserving)

Definition:

- Improving code's shape WITHOUT changing what it does, in small safe steps.
- Extract a component (split a big one), extract a custom hook (pull logic out of UI), remove duplication (DRY), rename for clarity.
- C#/.NET: like ReSharper/Roslyn "Extract Method" — same behavior, cleaner code.

✓ **Advantages:**

- Easier to read and test; low risk because each step is tiny and verified.

✗ **Disadvantages / watch-outs:**

- Verify after EACH step; never mix a refactor and a new feature in one commit.

💡 **Interview tip:**

- "Refactoring vs rewriting?" → refactor = small steps, behavior unchanged, always working; rewrite = throw away and rebuild = higher risk.

Example:

- `useEmployees()` — extract the fetch logic from a component into a hook.

Separation of Concerns / Service Layer

Definition:

- Keep jobs separate: components show UI, hooks hold logic, a service layer talks to the API.
- C#/.NET: exactly like a Service/Repository layer — controllers (components) never write raw HTTP.

✓ **Advantages:**

- API changes live in one file; components stay clean and easy to mock/test.

💡 **Interview tip:**

- "Why a service layer?" → one place for base URL, headers, error handling, and types; components don't repeat fetch code.

Example:

- `employeeApi.getAll()` — the component calls this, never `axios.get(url)` directly.

Loading / Error / Empty States

Definition:

- Every screen that fetches data must handle three extra states, not just success.
- "Empty" (no results) is a different message from "error" (request failed).

✓ Advantages:

- Users never see a blank or frozen screen — it looks professional.

✗ Disadvantages / watch-outs:

- Forgetting the empty state makes "no data" look like a broken page.

📄 Example:

- `if (loading) ...; if (error) ...; if (!data.length) ...` — guard in that order before rendering the list.

Whole-Plan Recall (Days 1–9)

Modern JavaScript / ES6+ (Day 1)

Definition:

- `let` / `const` (block scope), arrow functions, destructuring, and spread/rest — the everyday JS React leans on.
- Tip: `const` by default, `let` only when you reassign; arrow functions keep `this` for you.

📄 Example:

- `const { name, age } = emp` (destructure); `[...arr, x]` (spread copy).

TypeScript (Day 2)

Definition:

- JavaScript + types: `interface` / `type` describe shapes; generics reuse types safely.
- C#/.NET: like C# classes/interfaces + generics — the compile-time safety you already know.

📄 Example:

- `interface Employee { id: number; name: string }` — a typed shape.

Components, Props & JSX (Day 3)

Definition:

- A component is a function that returns JSX (HTML written in JS); props are its read-only inputs.
- C#/.NET: like a partial view / user control that takes parameters.

📄 Example:

- `function Card({ title }: { title: string }) { return <h2>{title}</h2>; }`

useState & Immutability (Day 4)

Definition:

- `useState` gives a component memory; update it through the setter with a NEW value.
- Use the functional form when the next value depends on the old one.

Example:

- `const [count, setCount] = useState(0); setCount(c => c + 1);`

useEffect & Data Fetching (Day 5)

Definition:

- `useEffect` runs side effects (like fetching) after render; the deps array controls when it re-runs.

✗ Disadvantages / watch-outs:

- Empty deps `[]` = run once; wrong deps = infinite loop or stale data.

Example:

- `useEffect(() => { load(); }, [])` — fetch on mount.

Controlled Forms & Validation (Day 6)

Definition:

- Controlled input = React state is the single source of truth; validate with a zod schema through react-hook-form.

💡 Interview tip:

- "Controlled vs uncontrolled?" → controlled = value in state (predictable, easy to validate); uncontrolled = value in the DOM, read via a ref (less code).

Example:

- `useForm({ resolver: zodResolver(schema) })` — a typed, validated form.

React Router (Day 6)

Definition:

- Client-side routing: map URLs to components; read URL params with `useParams`.
- C#/.NET: like MVC routes mapping a URL to an action — but in the browser, no page reload.

Example:

- `<Route path="/employees/:id" element={<Detail />} />` + `useParams()`.

Custom Hooks & Reusable Components (Day 7)

Definition:

- A custom hook is a `use...` function that packages reusable logic; a reusable component is configured through typed props.

✅ Advantages:

- DRY — test the logic once, keep the UI thin.

📄 Example:

- `function useDebounce(value: string, ms: number) { ... }` — share debounce logic.

Advanced React (Day 8)

Definition:

- Context (share state), useReducer (complex state), memo/useMemo/useCallback (skip needless work), lazy (load on demand), error boundaries (catch render crashes).

💡 Interview tip:

- "useMemo vs useCallback?" → useMemo caches a computed VALUE; useCallback caches a FUNCTION reference (`useCallback(fn)` $\approx$ `useMemo(()  $\Rightarrow$  fn)`).

❌ Disadvantages / watch-outs:

- Don't over-memoize — only when there's a real, measured cost.

📄 Example:

- `const total = useMemo(()  $\Rightarrow$  sum(items), [items])` — recompute only when items change.

Git/PR Workflow & Typed API Layer (Day 9)

Definition:

- Branch → commit → PR → review → merge; all API calls go through one typed axios service.
- C#/.NET: the same PR flow you use in .NET; the service layer mirrors your repository classes.

📄 Example:

- `git checkout -b feat/employee-search` — a feature branch before you start coding.